SPHERE WEB AUTOMATION

Public Login Script Step-by-Step Walkthrough

A control-by-control map of how `tests/public-login.spec.ts` verifies the Sphere login page — every mode toggle, modal, locator and edge case, with page-anatomy, state-machine and assertion-depth diagrams.

Script tests/public-login.spec.ts · 453 lines · 27 declarations → 29 tests in 10 groups

Target https://sphere.aastrika.org/public/login (no login required)

Run npm run run:login

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, and what it deliberately does not do
2	Configuration, helpers, credentials	The four helpers and why each exists
3	Page anatomy	Diagram — all 10 groups mapped onto the page
4	The three page states	Diagram — OTP mode, password mode, help modal
5	Assertion depth	Diagram — where every click test stops
6	Group 1 — Header	3 tests
7	Groups 2 & 3 — Welcome, Mode Toggle	2 + 3 tests
8	Group 4 — OTP login flow	4 tests
9	Group 5 — Password login flow	5 tests
10	Groups 6 & 7 — Create Account, Help modal	2 + 4 tests
11	Group 8 — Input validation edge cases	1 test, 10 payloads
12	Groups 9 & 10 — Responsive, Page health	3 + 2 tests
13	What green actually proves	The tally, and the login that is never asserted
14	Artifacts and run variants	Reports, console output, grep recipes
15	Failure modes and fixes	Troubleshooting reference

How to read this document

Sections 6 through 12 walk the script in file order, one table per `describe` group. Each row gives the **action**, the exact **locator**, and the **assertion** — with a marker for what the test can actually catch:

Marker	Meaning
hard	An `expect()` always runs. A missing or wrong element fails the test.
present-only	Asserts the control *exists*, then clicks it through `safeClick` and logs the outcome. A broken click or a wrong destination does not fail.
log	No `expect()` at all — the test records values to the console for a human to read.

That middle category is the defining shape of this suite, so Section 5 diagrams it and Section 13 tallies it.

1 What this script does

It verifies that the login page presents every control a returning user needs, in both of its input modes — and it is the source of truth for the selectors the rest of the project logs in with.

`/public/login` is the gate every returning learner passes through. It offers two ways in — a one-time code or a password — plus a help modal, a create-account escape hatch and a forgotten-password path. This spec walks all of it: each control, each mode, each modal, plus responsive rendering and page health.

Its second, quieter job

This file is where the project's login selectors are proven — `tests/helpers/login.ts` says so in its own header: *"Selectors mirror the ones verified in public-login.spec.ts."* So a change that breaks a locator here breaks every authenticated suite downstream — and this spec is the cheapest place to find out. That is a large part of its value, and it explains the file's shape: broad locator coverage, shallow outcome assertions.

What it deliberately does not do

No test in this file asserts that a login succeeded

The "Login" test fills real credentials, clicks the button, waits 2500 ms, then **logs** the resulting URL and any toast — there is no assertion about where the browser ended up. The same is true of "Get OTP", "Forgot Password?", "Create an Account" and the back arrow. This is by design (the suite owns no account whose state it can rely on), but it means a broken authentication backend would not turn this suite red. That assertion lives in `tests/login-check.spec.ts`.

The four things it verifies

Controls exist

- Both mode radios, Get OTP, Login
- Forgot Password, Create an Account
- Page title names Sphere or Aastrika

Modes behave

- Password mode reveals the password field
- Toggling back to OTP removes it entirely
- Password input is genuinely masked

Modal and health

- Help opens the chat-support modal
- Its X button really dismisses it

Entry points

Command	What it runs
`npm run run:login`	All 29 tests, headed, against staging. The full walkthrough this document describes.
`npm run report:ci`	This spec plus `PublicPage.spec.ts`, headless — the public-surface CI gate.
`npm run login:check`	**A different spec** — the one that actually asserts a successful sign-in with real credentials.

Two files share the same describe name

The outer block here is called `'Public Page - Complete Element Coverage'` — identical to the one in `PublicPage.spec.ts`. A `--grep` on that string matches both files. Grep an inner group name (`"Password Login Flow"`, `"Help Chat Support Modal"`) or pass the file path to target this spec alone.

2 Configuration, helpers, credentials

2.1 Target and navigation

```
// tests/public-login.spec.ts:5
const BASE_URL   = process.env.BASE_URL ?? 'https://sphere.aastrika.org';
const PUBLIC_URL = `${BASE_URL}/public/login`;

// :49 — runs before all 29 tests
await page.goto(PUBLIC_URL);
await page.waitForLoadState('networkidle');
```

Note the wait condition: `networkidle`, not `domcontentloaded`. The login page is a small Angular shell whose controls appear only once its bootstrap requests settle, so waiting for the network to go quiet is what makes the very first assertion in each test reliable. It also makes each test's setup slower than the public home page's, which is an acceptable trade at 29 tests.

Unlike `PublicPage.spec.ts`, this file does **not** normalise the origin — a `BASE_URL` with a trailing slash produces a double-slashed URL. In practice Angular's router tolerates it, but it is a difference worth knowing if you point the suite somewhere unusual.

2.2 The four helpers

Helper	Line	What it solves
`scrollAndClick`	:10	`scrollIntoViewIfNeeded()` then `click()`, unguarded. Used where a failure should surface.
`identifierInput`	:19	Returns the phone/email box. **The first `<input>` on this page is the OTP/Password radio**, which cannot be filled — so a naive `input.first()` throws. This helper filters to text, email, tel and untyped inputs before taking the first.
`helpControl`	:26	Returns the Help control. It is a clickable `<div>` wrapping a help-icon image, **not a `<button>`** — so `getByRole('button')` never finds it. Matches the exact text "Help", or any image with help in its alt.
`safeClick`	:35	Wraps `scrollAndClick` in try/catch. On success it logs `✅ Clicked: <label>`; on failure it logs a warning and **the test continues**. This one helper is why so much of the suite is present-only.

```
// :35 — a click that can never fail the test
async function safeClick(page, locator, label) {
  try {
    await scrollAndClick(page, locator);
    console.log(`✅ Clicked: ${label}`);
  } catch (err) {
    console.log(`⚠️  Could not click ${label}: ${err.message.split('\n')[0]}`);
  }
}
```

> **Two helpers encode real discoveries about this page**
> `identifierInput` and `helpControl` both exist because the obvious locator was wrong, and both carry a comment in the source explaining why. They are worth reading before writing any new test here: the first input is not the text box, and the Help control is not a button. Reuse the helpers rather than re-deriving the locators.

2.3 Test credentials

Value	Used in	Notes
`testcase003@yopmail.com`	OTP flow test 4, Password flow test 5	Hard-coded, not read from env. Whether this account still exists on the target environment is never asserted.
`Sunbird@123`	Password flow tests 2, 3, 5	The project's standard test password — the same one `create-account.spec.ts` sets.
`9876543210`	OTP flow test 1	A format sample, only ever typed and read back. Never submitted.

Because no test asserts the outcome of a submission, stale credentials do not fail the suite — they simply change what gets logged. That is consistent with the file's design, but it does mean the credentials can silently rot.

2.4 One thing this spec does not inherit from its sibling

> **No bilingual matching here**
> `PublicPage.spec.ts` routes every text assertion through a `tr()` helper because the portal serves Hindi by default. This file has no equivalent — every matcher is an English-only regex (`/login with otp/i`, `/get otp/i`, `/forgot password\??/i`). On an environment rendering the login page in Hindi, the text-based assertions would not match. The suite passes today because the environments it runs against serve this page in English; if that changes, the fix is the same `tr()` pattern the sibling file already implements.

3 Page anatomy

All 10 groups mapped onto the login page and its two overlay states.

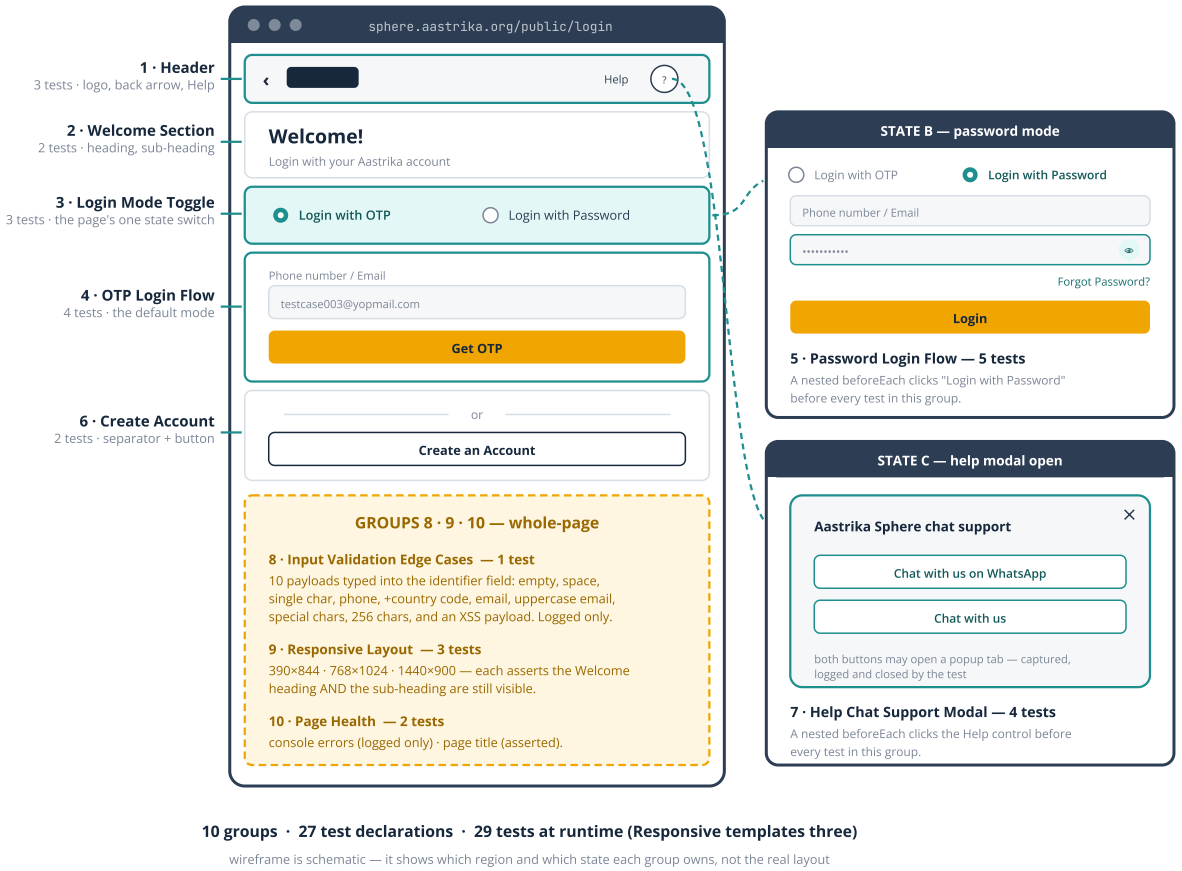


Figure 1 — Every `describe` group mapped to the control it covers. Groups 5 and 7 operate on states the page only reaches after a click, shown as insets.

4 The three page states

Every test starts in the same state. Two groups use a nested `beforeEach` to move somewhere else first.

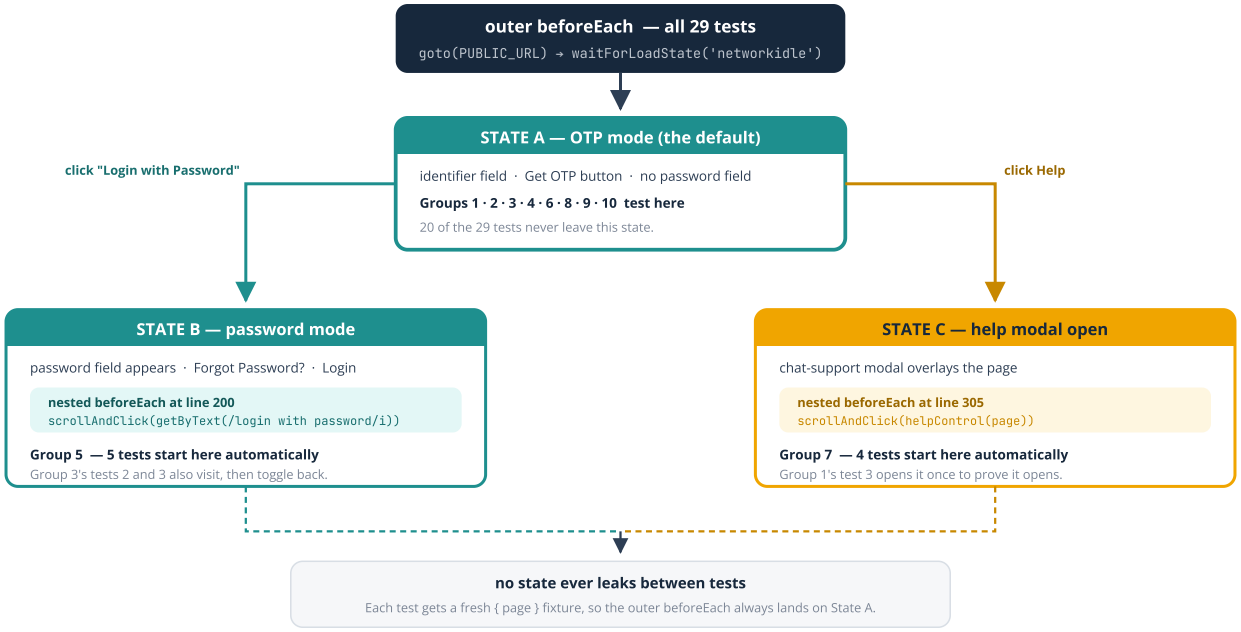


Figure 2 — Three states, two transitions. The nested `beforeEach` hooks are what let Groups 5 and 7 read as flat lists of element checks rather than repeating a setup click in every test.

Why the nested hooks matter

Without them, all five password tests and all four modal tests would each have to open their state by hand — nine repetitions of the same two lines, and nine places to update when the toggle's markup changes. The hooks put the setup in one place per group.

They also shift what a failure means. If `scrollAndClick(getByText(/login with password/i))` throws in the Group 5 hook, **all five tests in that group fail at once**, all with the same hook error. That is the correct signal — the mode toggle is broken, not five separate controls — but it is worth recognising when reading a report: five reds from one cause.

> **Group 3 is the one that tests the transition itself**
>
> Groups 5 and 7 *use* the transitions; Group 3 *tests* them. Its third test is the strongest assertion in the file about state — after toggling to password mode and back, it asserts `expect(count).toBe(0)` on `input[type="password"]`: the field must be gone from the DOM, not merely hidden. If Group 3 is green and Group 5 fails in its hook, the problem is the password screen, not the toggle.

5 Assertion depth

Nine tests click something that navigates or submits. Understanding where each one stops is the key to reading this suite.

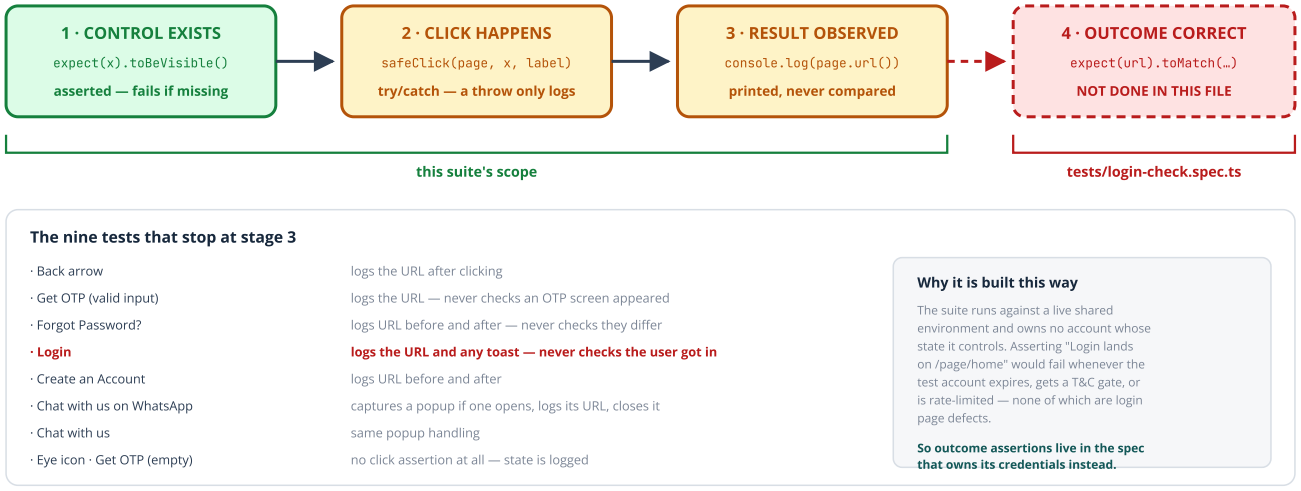


Figure 3 — Every click test in this file reaches stage 3 and stops. Stage 4 is covered by `login-check.spec.ts`, which takes its credentials from the environment and asserts the browser left `/public/`.

What this means in practice

A green run of this suite says: **every control is present, correctly labelled, and clickable without throwing.** It does not say the login works. When triaging an auth incident, this file tells you whether the page is intact; `login-check.spec.ts` tells you whether anyone can get in. Run both, in that order.

6 Group 1 — Header

`Header` · 3 tests · line 58

What it proves: the page is branded, offers a way back, and its help channel opens. Two of the three locators here exist only because the obvious approach failed.

#	Action	Locator	Assertion
1	**Logo is visible** Four alternatives in one locator.	`img[alt*="logo" i], img[alt*="aastrika" i], [class*="logo"] img, ws-app-public-nav-bar img → .first()`	hard `toBeVisible()`. Covers alt text, a logo-class wrapper, and the Angular nav-bar component's own image.
2	**Back arrow is visible and clickable** Matched by its Material icon ligature text.	`getByRole('button').filter({ hasText: /chevron_left|arrow_back/i }) .or(getByRole('button').first() ) → .first()`	present-only — visibility is asserted; the click goes through `safeClick` and the resulting URL is logged after 800 ms.
3	**Help opens the chat-support modal** The strongest test in the group — two hard assertions.	`helpControl(page) → getByText('Help', { exact: true }).or(img[alt*="help" i]) then getByText(/aastrika sphere chat support/i)`	hard ×2 — the control is visible, and after clicking it the modal heading is visible. A modal that fails to open fails this test.

The Material icon trick in test 2

Angular Material renders icons as **ligature text** inside a `<mat-icon>`: the element literally contains the string `chevron_left`, which the icon font draws as an arrow. So the back button has no accessible name a human would recognise — but it does have matchable text:

```
page.getByRole('button').filter({ hasText: /chevron_left|arrow_back/i })
    .or(page.getByRole('button').first())    // fallback: the first button on the page
    .first();
```

The `.or()` fallback is a pragmatic hedge — if the icon is ever swapped for an SVG or an aria-label, the locator degrades to "the first button", which on this page is the back control. It keeps the test running, at the cost of possibly asserting on the wrong button if the header is ever reordered.

Why `helpControl` is a helper, not an inline locator

The source comment is explicit: the Help control is *a clickable div with a "help-icon" img, not a* `<button>`, so `getByRole('button')` will not find it. Because Group 7's `beforeEach` also needs to open the modal, the locator is factored out — one definition, two call sites, and a comment recording the discovery so nobody re-derives it.

> **Test 3 is the header's real assertion**
>
> Tests 1 and 2 confirm the page is dressed correctly. Test 3 confirms something *works* — a click produces a modal, asserted, not logged. It is one of only two places in the whole file where a click's consequence is hard-asserted; the other is the modal's close button in Group 7.

7 Groups 2 & 3 — Welcome and Mode Toggle

Group 2 — Welcome Section 2 tests · line 96

The page's two copy elements. Both are plain hard visibility checks, and both are reused as the assertion in all three responsive tests later.

#	Element	Locator	Assertion
1	**"Welcome!" heading**	`getByText(/^welcome!?$/i).first()`	hard visible. Anchored at both ends with an optional `!`, so it cannot match a longer sentence containing the word.
2	**"Login with your Aastrika account" sub-heading**	`getByText(/login with your aastrika account/i).first()`	hard visible.

The anchoring in test 1 is the detail worth copying. `/^welcome!?$/i` matches the heading and nothing else; an unanchored `/welcome/i` would also match "Welcome back", a toast, or any paragraph mentioning the word — and would then assert on whichever came first in the DOM.

Group 3 — Login Mode Toggle 3 tests · line 114

The page's only state switch, and the group that tests it directly rather than merely using it.

#	Action	Locator	Assertion
1	**"Login with OTP" option** The default mode.	`getByText(/login with otp/i).first()`	hard visible. Note the test's *name* says "selected by default" but the body only asserts visibility — the radio's checked state is never read.
2	**Password mode reveals the password field** Click the radio, then prove the field appeared.	`getByText(/login with password/i).first()` then `getByLabel(/enter password/i).or(input[type="password"]).first()`	hard ×2 — the option is visible, and after a 400 ms settle the password field is visible.
3	**Toggling back to OTP removes the field** Password → OTP, then count what is left.	`click /login with password/i, wait 300 ms` `click /login with otp/i, wait 300 ms` then `input[type="password"].count()`	hard `expect(count).toBe(0)` — the field must be **absent from the DOM**, not merely hidden.

Test 3 is the most precise assertion in the file

`toBe(0)` on a locator count is a stronger statement than `not.toBeVisible()`. A password input left in the DOM but hidden with CSS would satisfy "not visible" while still holding the user's typed password in a live form control — exactly the kind of thing that leaks a credential into a subsequent request. Counting to zero rules that out.

A gap worth naming in test 1

"Login with OTP radio is selected by default" asserts only that the label is on screen. The actual default — that OTP mode is pre-selected, so a user who types an identifier and presses Get OTP gets an OTP — is not checked. Reading the radio's checked state would be a one-line addition, and it would make the test match its own title.

8 Group 4 — OTP login flow

`OTP Login Flow` · 4 tests · line 151 · tests State A, the default mode

What it proves: the identifier field accepts both kinds of identity Sphere allows, and the Get OTP button is present and clickable. Whether an OTP is actually sent is not checked.

#	Action	Locator	Assertion
1	**Identifier accepts phone and email** Types each, reads it back. The strongest test in the group.	`getByLabel(/phone number.*email/i) .or(input[type="text"], input[type="email"], input[type="tel"]) → .first()`	hard ×3 — visible; `toHaveValue('9876543210')`; then cleared and `toHaveValue('testcase003@yopmail.com')`. One field, both identity types.
2	**"Get OTP" button is visible**	`getByRole('button', { name: /get otp/i })`	hard `toBeVisible()`.
3	**"Get OTP" disabled when empty** Clears the field, then reads the button's state.	`identifierInput(page).fill('') then getOtpBtn.isDisabled().catch(() ⇒ false)`	log — prints `Empty-state — Get OTP disabled: true/false`. **No `expect()`.** The gate can be broken and this test still passes.
4	**"Get OTP" clickable with valid input** Fills a real address and submits.	`identifierInput(page).fill('testcase003@yopmail.com') then safeClick(getOtpBtn)`	present-only — the click is wrapped in `safeClick`, then the URL is logged after 1500 ms. No assertion that an OTP screen appeared.

Why test 1 uses a different locator from tests 3 and 4

Test 1 leads with `getByLabel(/phone number.*email/i)` — the accessible label, which is the most semantically correct handle — and falls back to the type-based selector. Tests 3 and 4 call the `identifierInput` helper, which is type-based only. Both resolve to the same field on today's markup. Using the helper everywhere would be marginally more consistent and would put the "first input is the radio" knowledge in exactly one place.

Test 3 is the clearest hardening opportunity in the file

```
// :175 — as written, this cannot fail
const disabled = await getOtpBtn.isDisabled().catch(() ⇒ false);
console.log(`✅ Empty-state — Get OTP disabled: ${disabled}`);

// what it would take to make it real:
expect(disabled).toBe(true);
```

Whether the button is gated on input is genuine product behaviour with a definite correct answer, and unlike the navigation tests it needs no account and no backend. The ✅ in the log line is also slightly misleading — it prints regardless of what the value turned out to be.

> **What test 4 does and does not cover**
> It proves the submit path is reachable: a valid identifier can be typed and the button accepts a click without throwing. It does not prove an OTP was requested, sent, or received — and it will log a perfectly normal-looking URL whether the request succeeded or returned a 500. End-to-end OTP delivery is covered by `create-account.spec.ts` and `OTP.spec.ts`, which read the real mailbox.

9 Group 5 — Password login flow

`Password Login Flow` · 5 tests · line 198 · nested beforeEach → State B

```
// :200 — runs before each of the five tests below
test.beforeEach(async ({ page }) => {
  await scrollAndClick(page, page.getByText(/login with password/i).first());
  await page.waitForTimeout(400);
});
```

Every test here therefore starts with the password field on screen. The hook is unguarded — if the toggle breaks, all five tests fail together with the same hook error.

#	Action	Locator	Assertion
1	**Identifier still visible in password mode** Switching modes must not hide the identity field.	`getByLabel(/phone number.*email/i).or(input.first()) → .first()`	hard visible.
2	**Password field masks input** Fills it, then reads the type attribute.	`input[type="password"].first()` then `getAttribute('type')`	hard ×2 — visible, and `expect(type).toBe('password')`. A field that renders the password in clear text fails here.
3	**Eye icon toggles visibility** Attempts the reveal control.	`mat-icon:has-text("visibility"), mat-icon:has-text("visibility_off"), button[aria-label*="password" i] → .first()`	log — presence is logged, and if present it is clicked and the input's type is logged after. **No `expect()` anywhere.**
4	**"Forgot Password?" link**	`getByText(/forgot password\??/i).first()`	present-only — visibility asserted; `safeClick`, then URL before and after are both logged but never compared.
5	**"Login" button submits** Fills real credentials and clicks.	`identifierInput ← testcase003@yopmail.com` `input[type="password"] ← Sunbird@123` `getByRole('button', { name: /^login$/i })` then `mat-snack-bar-container, [role="alert"], .toast, .error-message`	present-only — visibility asserted; `safeClick`, then after 2500 ms the URL and any toast text are logged. **Nothing about the outcome is asserted.**

Test 2 is doing real security work

Asserting `type === 'password'` after filling the field confirms the browser's native masking is active. It is a small check with a real failure mode behind it: a field whose type was switched to `text` — by a mis-wired reveal toggle, say — would show the password on screen and, worse, expose it to any screen-capture or screen-sharing in progress. This is one of the few assertions in the file that would catch a genuine defect rather than a missing element.

Test 3's known weak point

```
if (eyeVisible) {
  await scrollAndClick(page, eyeIcon);
  // reads input index 1 — assumes the password box is the second input
  const afterType = await page.locator('input').nth(1).getAttribute('type').catch(() => null);
```

```
  console.log(`   Password input type after toggle: ${afterType}`);
}
```

Two things stand out. It reads `input` by raw index — and index 1 is a guess that holds only while the DOM order is radio, identifier, password. And the result is printed, not asserted, though the whole point of the toggle is that the type flips `password` → `text`: a definite, checkable outcome that would pair naturally with test 2.

> **Test 5 is where "present-only" matters most**
> This test types a real email and a real password into the real login form of a live environment and clicks Login. Whatever happens next — success, "invalid credentials", a rate-limit, a 500 — the test passes and prints it. The captured toast is genuinely useful when reading a run by hand, but nothing gates on it. If you want this suite to notice that logins have stopped working, this is the test to change; if you want that coverage without changing this file, run `npm run login:check`, which asserts the browser left `/public/`.

10 Groups 6 & 7 — Create Account and Help modal

Group 6 — Create Account

2 tests · line 277

The escape hatch for a visitor who does not yet have an account.

#	Element	Locator	Assertion
1	**"or" separator**	`getByText(/^or$/i).first()`	hard visible. Anchored — an unanchored `/or/i` would match almost every word on the page.
2	**"Create an Account" button**	`getByRole('button', { name: /create an account/i }).or(getByText(/create an account/i)).first()`	present-only — visibility asserted; `safeClick`, URLs before and after logged.

The role-then-text `.or()` is the same hedge used for the Help control: match the accessible button if there is one, fall back to the text node if it is a styled div.

Group 7 — Help Chat Support Modal

4 tests · line 303 · nested beforeEach → State C

```
// :305 — opens the modal before each of the four tests
await scrollAndClick(page, helpControl(page));
await page.waitForTimeout(500);
```

#	Action	Locator	Assertion
1	**Modal heading**	`getByText(/aastrika sphere chat support/i)`	hard visible — confirms the hook's click landed.
2	**"Chat with us on WhatsApp"** May open a new tab.	`getByText(/chat with us on whatsapp/i).first()` + `page.waitForEvent('popup', { timeout: 3000 })`	present-only — visibility asserted; the popup (if any) has its URL logged and is closed. If none opens, logs "may have navigated in-page".
3	**"Chat with us"** Must not match the hidden third-party widget.	`getByRole('button', {name:/^chat with us$/i}).or(getByText(/^chat with us$/i))` **`.filter({ visible: true })`** → `.first()`	present-only — same popup handling as test 2.
4	**Close (X) dismisses the modal** The group's real assertion.	`button:has(mat-icon:has-text("close")), [aria-label*="close" i], button:has-text("×")` → `.first()`	hard ×2 — close button visible, and after clicking, `expect(stillOpen).toBe(false)` on the modal heading.

The `.filter({ visible: true })` in test 3

The source comment explains it: a hidden third-party chat widget elsewhere on the page also contains the text "chat with us". Without the filter, `.first()` could resolve to that invisible element, and the test would fail on a modal button that is perfectly fine. Anchoring the regex (`/^chat with us$/i`) narrows it further so it cannot match the longer WhatsApp label from test 2. Two constraints, two distinct false matches ruled out.

Popup handling, and why it cannot fail

```
const popupPromise = page.waitForEvent('popup', { timeout: 3000 }).catch(() => null);
await safeClick(page, waBtn, 'Chat with us on WhatsApp');
const popup = await popupPromise;
```

```
if (popup) { console.log(popup.url()); await popup.close(); }
else       { console.log('   No popup detected'); }
```

The listener is registered *before* the click — a popup opening faster than the listener attaches would be missed. The `.catch(() ⇒ null)` turns the 3 s timeout into a null rather than a rejection, so both branches are normal outcomes.

> **Test 4 is the second of only two hard-asserted consequences in the file**
> Alongside "Help opens the modal" in Group 1, this is the other place where a click's effect is asserted rather than logged. Together they bracket the modal: it opens when asked, and it closes when dismissed. A modal that cannot be dismissed traps a user on a page they were trying to log in from — worth the hard assertion.

11 Group 8 — Input validation edge cases

`Input Validation Edge Cases` · 1 test, 10 payloads · line 372 · log — zero assertions

What it does: types ten different values into the identifier field, reads back what the field kept, and records whether Get OTP was enabled for each. It is a **reconnaissance** test — it produces a table of observed behaviour, not a verdict.

#	Payload	Label in the log	What it probes
1	`'' (empty)`	empty	The baseline — is the submit gated at all?
2	`' ' (single space)`	single space	Is whitespace trimmed, or treated as content that unlocks the button?
3	`'a'`	single char	Is there a minimum length?
4	`'1234567890'`	10-digit phone	The standard Indian mobile format.
5	`'+919876543210'`	phone with country code	Is the `+` prefix accepted, stripped, or rejected?
6	`'test@example.com'`	standard email	The other valid identity type.
7	`'TEST@EXAMPLE.COM'`	uppercase email	Is case normalised? Emails are case-insensitive in the domain part.
8	`'!@#$%^&*()'`	special chars	Does a nonsense string enable submission?
9	`'a'.repeat(256)`	very long (256 chars)	Is there a maxlength? The log truncates the echo to 40 chars.
10	`'<script>alert(1)</script>'`	XSS payload	Is the input escaped, stripped, or stored verbatim?

The loop

```
for (const c of cases) {
  await input.fill('');
  await input.fill(c.val);
  await page.waitForTimeout(150);
  const actual  = await input.inputValue();                    // what the field KEPT
  const enabled = await getOtpBtn.isEnabled().catch(() => true);  // did the gate open?
  const preview = actual.length > 40 ? `${actual.slice(0, 40)}...` : actual;
  console.log(`   • ${c.label.padEnd(25)} → accepted="${preview}" | Get OTP enabled=${enabled}`);
}
```

Two observations per payload: what survived the field's own filtering, and whether the button unlocked. The `padEnd(25)` aligns the output into a readable column, which is the point — this test's product is a table a human scans.

What the output looks like

```
   • empty                     → accepted="" | Get OTP enabled=false
   • single space              → accepted=" " | Get OTP enabled=true
   • single char               → accepted="a" | Get OTP enabled=true
   • 10-digit phone            → accepted="1234567890" | Get OTP enabled=true
   • phone with country code   → accepted="+919876543210" | Get OTP enabled=true
   • standard email            → accepted="test@example.com" | Get OTP enabled=true
   • special chars             → accepted="!@#$%^&*()" | Get OTP enabled=true
   • XSS payload               → accepted="<script>alert(1)</script>" | Get OTP enabled=true
```

Illustrative shape — the real values depend on the build under test.

Reading this test correctly

A row showing `enabled=true` for `single space` or `special chars` is a finding — it means the client accepts an identifier that can never match an account, and the user learns that only after a round trip. But this test **will not tell you**: it prints the row and passes. The output is evidence for a human, and it is only worth generating if someone reads it.

The `<script>` row is a probe, not an XSS test. Seeing the payload echoed back into the input's value is expected and harmless — a real check would confirm the string is escaped where it is *rendered*, and nothing here does that. Treat it as a note that the field does no input sanitisation, which is normal for a text box.

If you want this to gate anything

The cheapest meaningful assertion is on rows 1 and 2 — empty and whitespace-only must not enable Get OTP. Both are unambiguous, both need no backend, and both are already being measured. Everything else in the list is genuinely a judgement call about product intent, which is why logging is the honest default for them.

12 Groups 9 & 10 — Responsive and Page health

Group 9 — Responsive Layout 3 tests · line 407

One templated test per viewport. Unlike the signup suite's responsive block, these do not build new browser contexts — they resize the existing page, which is cheaper and sufficient for a layout smoke check.

Viewport	Size	Assertion
`Mobile (iPhone 12)`	`390 × 844`	hard ×2 per test — after `setViewportSize` and a 400 ms settle, both the `/^welcome!?$/i` heading and the `/login with your aastrika account/i` sub-heading must still be visible.
`Tablet (iPad)`	`768 × 1024`	
`Desktop`	`1440 × 900`	

```
for (const vp of viewports) {
  test(`Renders on ${vp.name}`, async ({ page }) => {
    await page.setViewportSize({ width: vp.w, height: vp.h });
    await page.waitForTimeout(400);
    await expect(page.getByText(/^welcome!?$/i).first()).toBeVisible();
    await expect(page.getByText(/login with your aastrika account/i).first()).toBeVisible();
  });
}
```

This is why the file declares 27 tests but runs 29 — the loop templates three.

Scope, stated honestly

Asserting the two headings is a reachability check: it proves the page did not collapse or clip its copy at 390px. It says nothing about the form controls at that width — whether the Login button is reachable without horizontal scrolling, whether tap targets are large enough, whether the mode radios still fit on one line. Extending the assertion to the identifier field and the primary button would cost two lines and cover the part of the page users actually have to operate.

Group 10 — Page Health 2 tests · line 430

#	Check	Mechanism	Assertion
1	**No console errors on load**	Attach a `console` listener, re-`goto`, wait for networkidle + 1000 ms, collect `msg.type() === 'error'`	log — prints the count and the first five messages (truncated to 200 chars each), then logs either "No console errors" or "⚠️ Console errors present". **No `expect()`.**
2	**Page title**	`await page.title()`	hard `toMatch(/aastrika|sphere/i)`. Cheap, and a real canary for a wrong shell or an error page served with a 200.

The console check here is weaker than its sibling

`PublicPage.spec.ts` runs the same check but **asserts** — it filters out `Failed to load resource` noise and then requires the remaining list to be empty, so a genuine JavaScript exception fails the suite. This file collects the same data and only prints it. Copying the sibling's two lines would make an uncaught exception on the login page visible in the exit code rather than buried in the log:

```
const jsErrors = errors.filter((e) => !/Failed to load resource/i.test(e));
expect(jsErrors).toEqual([]);
```

13 What green actually proves

24 of 29 tests carry a real assertion. Five carry none. And none of the 29 asserts that anyone can log in.

Group	Tests	Hard	Present-only	Log	What is actually guaranteed
1 · Header	3	**2**	1	0	Logo, and that Help opens the modal. The back arrow's destination is not checked.
2 · Welcome	2	**2**	0	0	Fully guaranteed.
3 · Mode Toggle	3	**3**	0	0	Fully guaranteed — including that the password field leaves the DOM.
4 · OTP Login Flow	4	**2**	1	1	Identifier accepts phone and email; Get OTP exists. The empty-state gate and the submit are not asserted.
5 · Password Login Flow	5	**2**	2	1	Identifier stays visible; password is genuinely masked. Eye icon, Forgot Password and Login outcomes are not asserted.
6 · Create Account	2	**1**	1	0	The separator. The button's destination is not checked.
7 · Help Modal	4	**2**	2	0	The modal's heading and that X dismisses it. Both chat buttons are present-only.
8 · Edge Cases	1	**0**	0	1	Nothing. Produces a readable table of ten observed behaviours.
9 · Responsive	3	**3**	0	0	Both headings visible at all three widths. Form controls not checked.
10 · Page Health	2	**1**	0	1	The page title. Console errors are logged, not asserted.
Total	**29**	**18**	**7**	**4**	**18 tests gate a build; 7 assert presence but not outcome; 4 only report.**

The one sentence version

A green run means the login page is intact and operable

Every control is present and correctly labelled, both input modes work, the password is masked, the help modal opens and closes, and the page renders from 390px to 1440px. That is genuinely valuable — it is the fastest way to know a deploy did not break the gate every returning user passes through, and it is the check that protects the selectors every authenticated suite depends on.

A green run does not mean anyone can log in

No assertion in this file examines where a click led. Run `npm run login:check` for that — it drives the same page through `tests/helpers/login.ts` with credentials from `LOGIN_ID` / `LOGIN_PASSWORD`, and asserts the browser left `/public/` for an authenticated route (`/page/home` , `/app/new-tnc` , or another `/app/…` page).

Where hardening would pay, in order

1. **Get OTP disabled when empty** (Group 4, test 3). One line: `expect(disabled).toBe(true)` — definite behaviour, no backend needed, already measured.
2. **Console errors** (Group 10, test 1). Two lines, copied verbatim from `PublicPage.spec.ts` . Turns a buried log line into a build signal.

3. **Eye icon toggle** (Group 5, test 3). Assert the type flips `password` → `text`, using the typed locator rather than `input.nth(1)`.
4. **OTP mode selected by default** (Group 3, test 1). Read the radio's checked state so the test matches its own name.
5. **Empty and whitespace rows** in the edge-case loop. Assert those two; leave the other eight as observation.

The navigation tests are best left alone. Asserting destinations on a live shared environment would fail this suite for reasons that are not login-page defects — precisely the trade `login-check.spec.ts` exists to take, with credentials it owns.

14 Artifacts and run variants

14.1 What a run leaves behind

Artifact	Contents
`test-results/`	A screenshot for every one of the 29 tests (`screenshot: 'on'`). Because Groups 5 and 7 run in non-default states, the set includes shots of password mode and the open modal.
`playwright-report/` · `video.webm`	The HTML report; video is kept only for failures.
`console output`	**The most informative artifact for this suite.** Every clicked destination, every popup URL, the server's response toast, the ten edge-case rows, and the console-error list. Seven of the 29 tests put their only real finding here.

14.2 A healthy run, abridged

```
✅ Page loaded: https://sphere.aastrika.org/public/login
   URL after back click: https://sphere.aastrika.org/public/home
✅ Help modal opened with heading "Aastrika Sphere chat support"
✅ Switching to Password mode reveals the password field
✅ Toggled back to OTP — password fields on screen: 0
✅ Identifier field accepted phone number / accepted email
✅ Empty-state — Get OTP disabled: true
✅ Password field present, input type="password" (masked)
✅ Clicked: Login
   URL after Login click: https://sphere.aastrika.org/public/login
   Server response: "Invalid username or password"
   Opened popup URL: https://wa.me/91…
   • empty                    → accepted="" | Get OTP enabled=false
   • XSS payload              → accepted="<script>alert(1)</script>" | Get OTP enabled=true
✅ Welcome + sub-heading visible at Mobile (390x844)
   Console errors captured: 0
```

Note the highlighted line: a failed login prints its server message and the run still passes — exactly the kind of finding this suite surfaces but does not act on.

14.3 Run variants

```
# the whole suite, headed, against staging
npm run run:login

# one group — use an INNER name; the outer describe collides with PublicPage.spec.ts
npx playwright test tests/public-login.spec.ts --grep "Password Login Flow"
npx playwright test tests/public-login.spec.ts --grep "Help Chat Support Modal"
npx playwright test tests/public-login.spec.ts --grep "Input Validation"

# point at another environment
cross-env BASE_URL=https://portal-staging.example.org \
  npx playwright test tests/public-login.spec.ts --headed

# headless, with the public page suite — the CI gate
npm run report:ci

# the suite that actually asserts a login succeeds
npm run login:check

# report / trace
```

```
npx playwright show-report
npx playwright test tests/public-login.spec.ts --trace on
```

15 Failure modes and fixes

Symptom	Most likely cause	What to do
All 29 tests fail	The page never loaded, or Playwright was run from the repo root instead of `SphereWebAutomation\`.	Check the URL printed by `beforeEach`. Run `--grep "Page Health"` — a failing title test confirms the page itself is wrong.
All five Password Login Flow tests fail with the same error	The group's `beforeEach` could not click "Login with Password" — one cause, five reds.	Check whether Group 3 also failed. Green Group 3 plus red Group 5 hook means the toggle works but the password screen does not render.
All four Help Modal tests fail together	Same shape — the Group 7 hook could not open the modal.	Check Group 1 test 3, which opens the modal independently. If that is also red, the Help control moved; fix `helpControl()` at line 26 and both groups recover.
"Cannot fill element" on the identifier field	Something resolved to the OTP/Password radio — the first `<input>` on the page.	Use `identifierInput(page)`. This is the exact bug the helper at line 19 was written to prevent.
"Chat with us" test fails on a visible button	The locator resolved to the hidden third-party chat widget whose text also contains that phrase.	Keep `.filter({ visible: true })` and the anchored `/^chat with us$/i`. Both constraints are load-bearing.
Back arrow test asserts the wrong button	The `.or(getByRole('button').first())` fallback engaged because the Material ligature text changed.	Check the icon's rendered text in the DOM and add it to the `/chevron_left|arrow_back/i` alternation rather than relying on the positional fallback.
Every text assertion fails, page looks fine	The environment is serving this page in Hindi. This file has no bilingual matching.	Port the `tr()` helper and `HI` map from `PublicPage.spec.ts` and route the text matchers through it.
Login logs "Invalid username or password"	The hard-coded `testcase003@yopmail.com` no longer exists on the target environment.	Not a test failure — the suite stays green. Mint a fresh account (`npm run mint:fresh-user`) and update the literal, or move the credentials to env vars.
Flaky first assertion in a test	The Angular shell had not finished painting when the assertion ran.	`beforeEach` already waits for `networkidle`. If a specific control is still racing, give that one assertion an explicit timeout rather than adding a blanket `waitForTimeout`.
A `--grep` runs far more tests than expected	You grepped the outer describe name, which is identical in `PublicPage.spec.ts`.	Grep an inner group name, or pass the file path explicitly.

The one rule to keep

This file's locators encode hard-won knowledge about a page whose controls are not what they look like — the first input is a radio, Help is not a button, the back arrow is icon ligature text, and a hidden widget shadows the chat button. **Reuse the helpers rather than re-deriving the locators**, and when you add a check, decide deliberately whether it asserts or observes. Both are legitimate here; what is not legitimate is a test whose name promises more than its body delivers.